Department of Computer Science & Engineering

Microprocessor & Computer Architecture
UE24CS251B

MPCA Lab Programs - 5

Name: Dhriti Kamani	SRN: PES1UG24AM085	Section:B

1 **Write an ARM assembly language program to compute the sum of all elements in a 3×3 matrix stored in memory.**

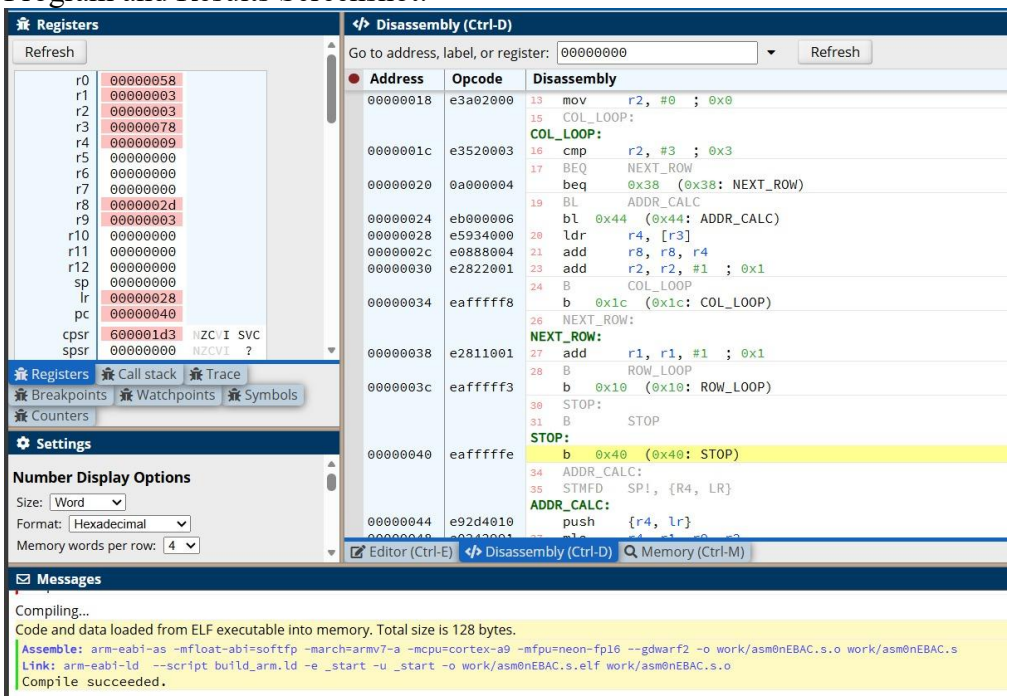
The matrix elements are stored in row-major order as a one-dimensional array. The program should:

- **Use nested loops to iterate over rows and columns.**
- **Compute the effective memory address of each matrix element using a separate subroutine.**
- **Load each matrix element from memory and accumulate the total sum in a register.**
- **Store or maintain the final sum in a designated register before program termination.**

The program must use:

- **A subroutine to calculate the address of a[i][j]**
- **Stack operations (STMFD, LDMFD) to preserve register values**
- **Multiply-Accumulate (MLA) for address computation**

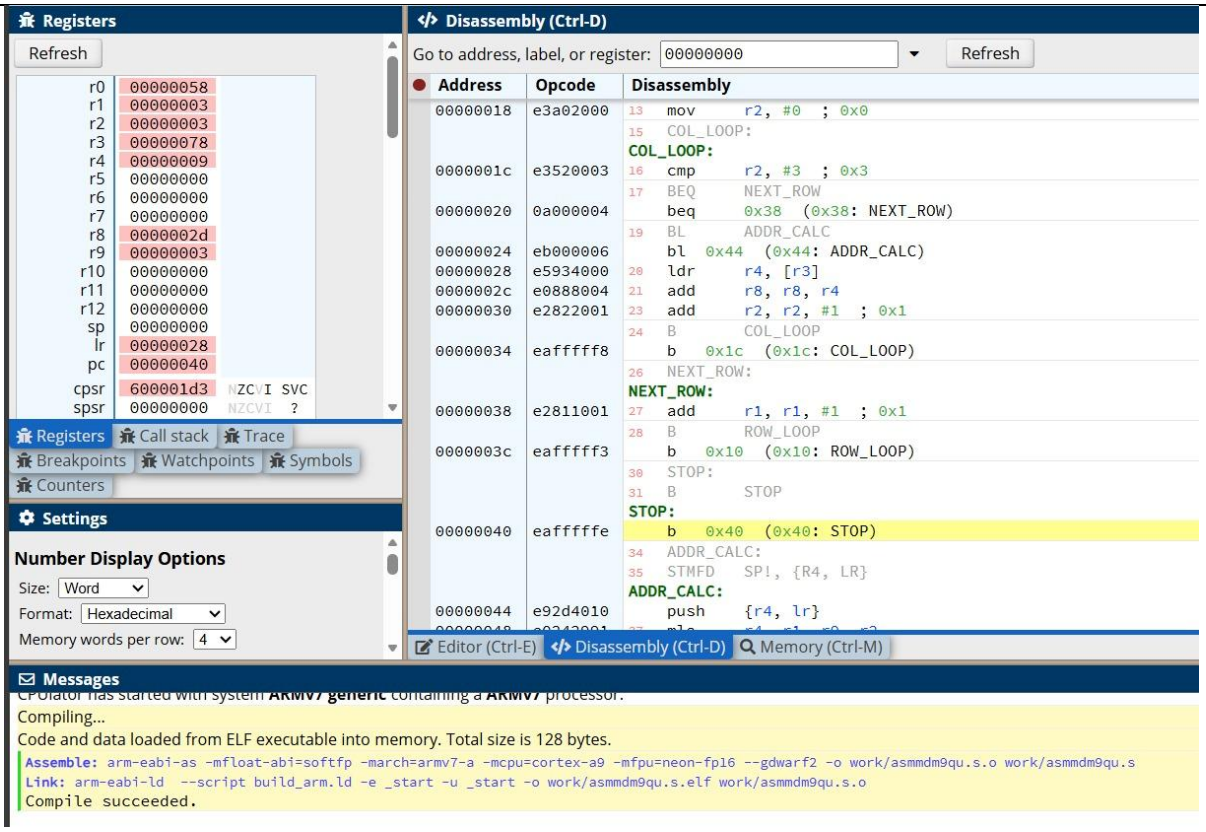
Program and Results Screenshot:




```
. global _start
_start:
        LDR     R0, =MAT
        MOV     R1, #0
        MOV     R8, #0
```

	MOV R9, #3
	ROW_LOOP:
	CMP R1, #3
	BEQ STOP
	MOV R2, #0
	COL_LOOP:
	CMP R2, #3
	BEQ NEXT_ROW
	BL ADDR_CALC
	LDR R4, [R3]
	ADD R8, R8, R4
	ADD R2, R2, #1
	B COL_LOOP
	NEXT_ROW:
	ADD R1, R1, #1
	B ROW_LOOP
	STOP:
	B STOP
	ADDR_CALC:
	STMFD SP!, {R4, LR}
	MLA R4, R1, R9, R2
	LSL R4, R4, #2
	ADD R3, R0, R4
	LDMFD SP!, {R4, PC}
	MAT:
	.word 1, 2, 3
	.word 4, 5, 6
	.word 7, 8, 9

2	**Write an ARM assembly program to transfer a 3×3 matrix stored in memory location A to memory location C. Use a subroutine to compute element addresses and stack operations to save and restore registers.** **The source matrix is stored as a one-dimensional array in row-major order. The program should:** • **Use nested loops to traverse rows and columns of the matrix.** • **Compute the effective address of each matrix element using a separate subroutine.** • **Load each element from the source matrix and store it into the destination matrix.** • **Use stack operations to preserve register values during subroutine calls.** • **Ensure that all matrix elements are copied correctly before program termination.** Program and Results Screenshot:

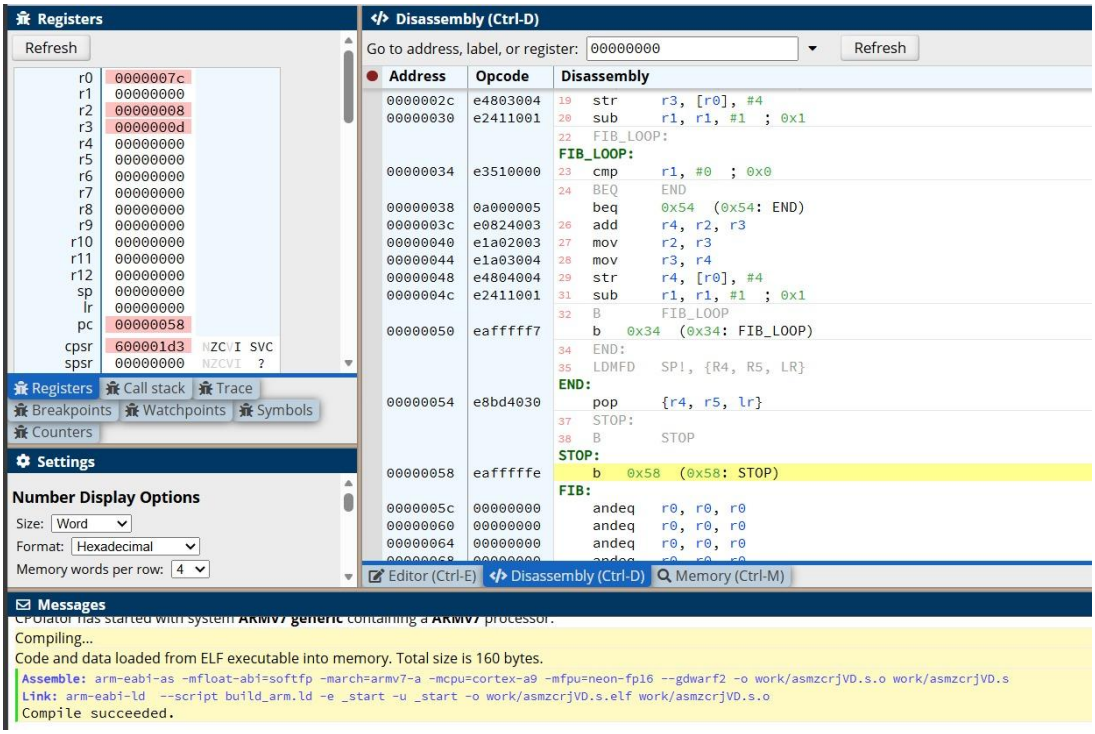



```
.global _start
_start:
        LDR     R0, =MAT
        MOV     R1, #0
        MOV     R8, #0
        MOV     R9, #3
ROW_LOOP:
        CMP     R1, #3
        BEQ     STOP
        MOV     R2, #0
COL_LOOP:
        CMP     R2, #3
        BEQ     NEXT_ROW
        BL      ADDR_CALC
        LDR     R4, [R3]
        ADD     R8, R8, R4
        ADD     R2, R2, #1
        B       COL_LOOP
NEXT_ROW:
        ADD     R1, R1, #1
        B       ROW_LOOP
STOP:
        B       STOP
ADDR_CALC:
        STMFD   SP!, {R4, LR}
        MLA     R4, R1, R9, R2
        LSL     R4, R4, #2
        ADD     R3, R0, R4
```

```
        LDMFD   SP!, {R4, PC}
MAT:
        .word   1, 2, 3
        .word   4, 5, 6
.word   7, 8, 9
```

3 **Generate the first N Fibonacci numbers and store them in memory while using stack operations for register preservation.**

Program and Results Screenshot:

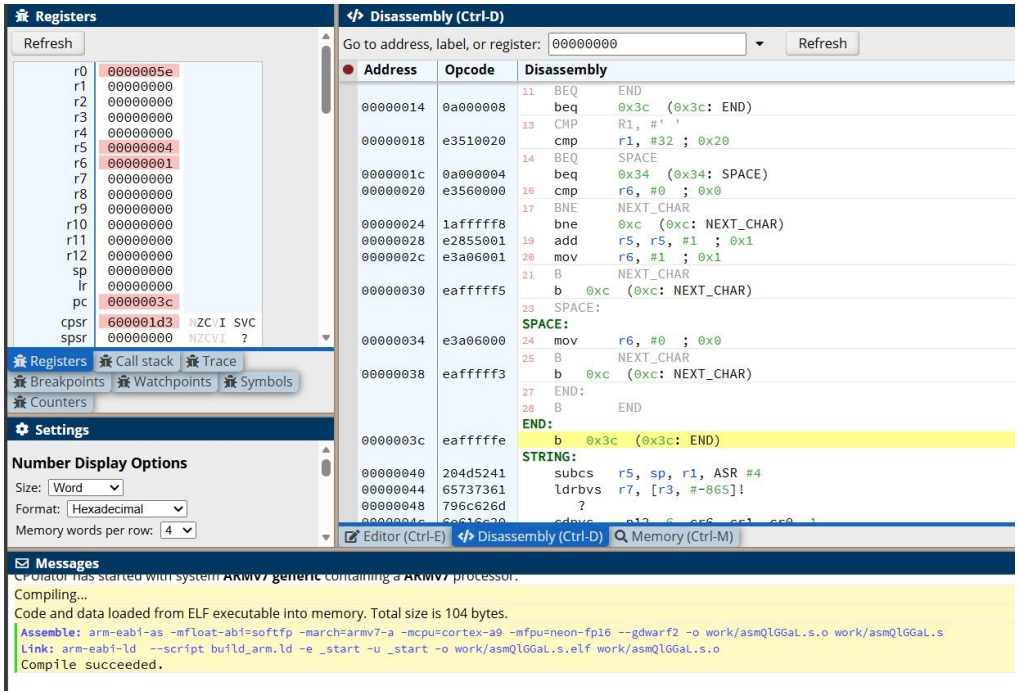



```
.global _start
_start:
        LDR     R0, =FIB
        MOV     R1, #8
        MOV     R2, #0
        MOV     R3, #1
        STMFD   SP!, {R4, R5, LR}
        CMP     R1, #0
        BEQ     END
        STR     R2, [R0], #4
        SUB     R1, R1, #1
        CMP     R1, #0
        BEQ     END
        STR     R3, [R0], #4
        SUB     R1, R1, #1
FIB_LOOP:
        CMP     R1, #0
        BEQ     END
        ADD     R4, R2, R3
        MOV     R2, R3
        MOV     R3, R4
        STR     R4, [R0], #4
```

```
        SUB     R1, R1, #1
        B       FIB_LOOP
END:
        LDMFD   SP!, {R4, R5, LR}
STOP:
        B       STOP
FIB:
        .space  64
```

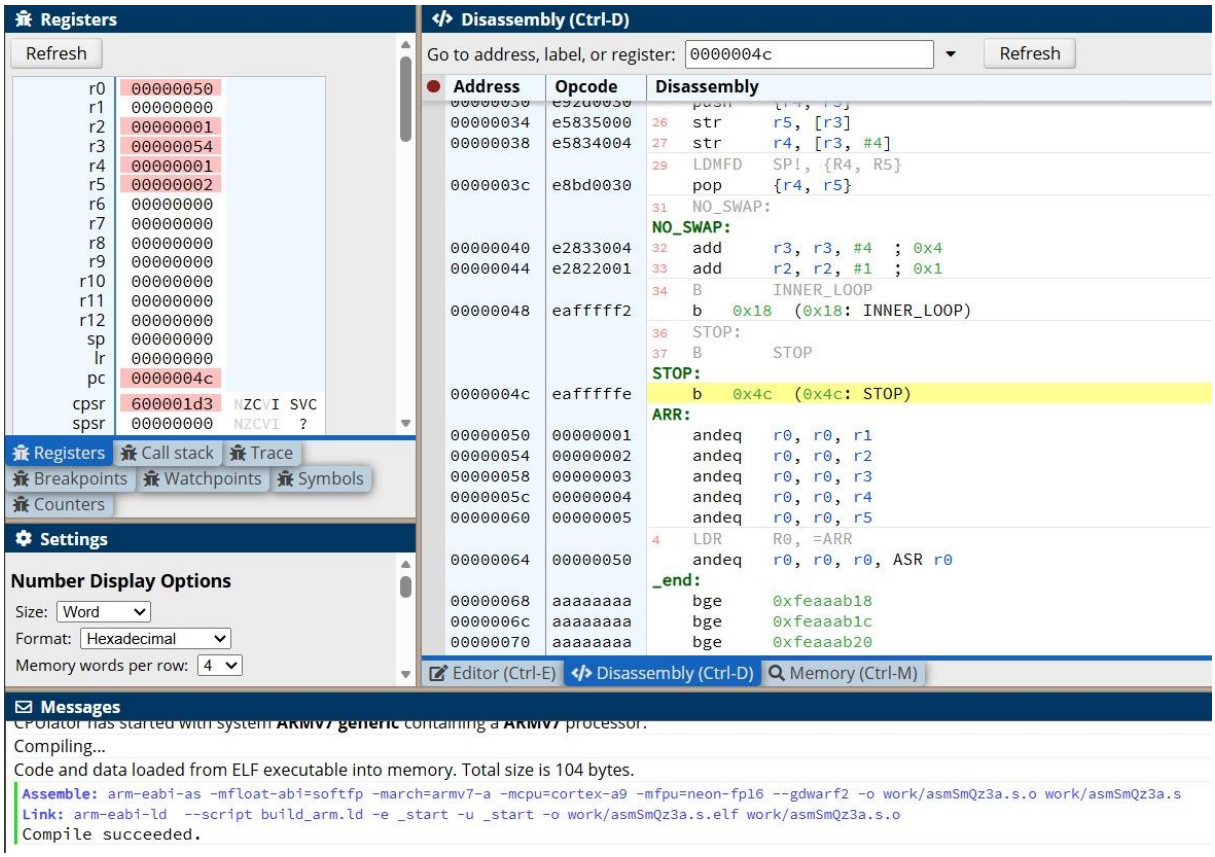
4 **Write an ARM ALP to count the number of words in a string.**

Program and Results Screenshot:




```
.global _start
_start:
        LDR     R0, =STRING
        MOV     R5, #0
        MOV     R6, #0
NEXT_CHAR:
        LDRB    R1, [R0], #1
        CMP     R1, #0
        BEQ     END
        CMP     R1, #' '
        BEQ     SPACE
        CMP     R6, #0
        BNE     NEXT_CHAR
        ADD     R5, R5, #1
        MOV     R6, #1
        B       NEXT_CHAR
SPACE:
        MOV     R6, #0
        B       NEXT_CHAR
END:
        B       END
```

	STRING: .asciz "ARM assembly language program"

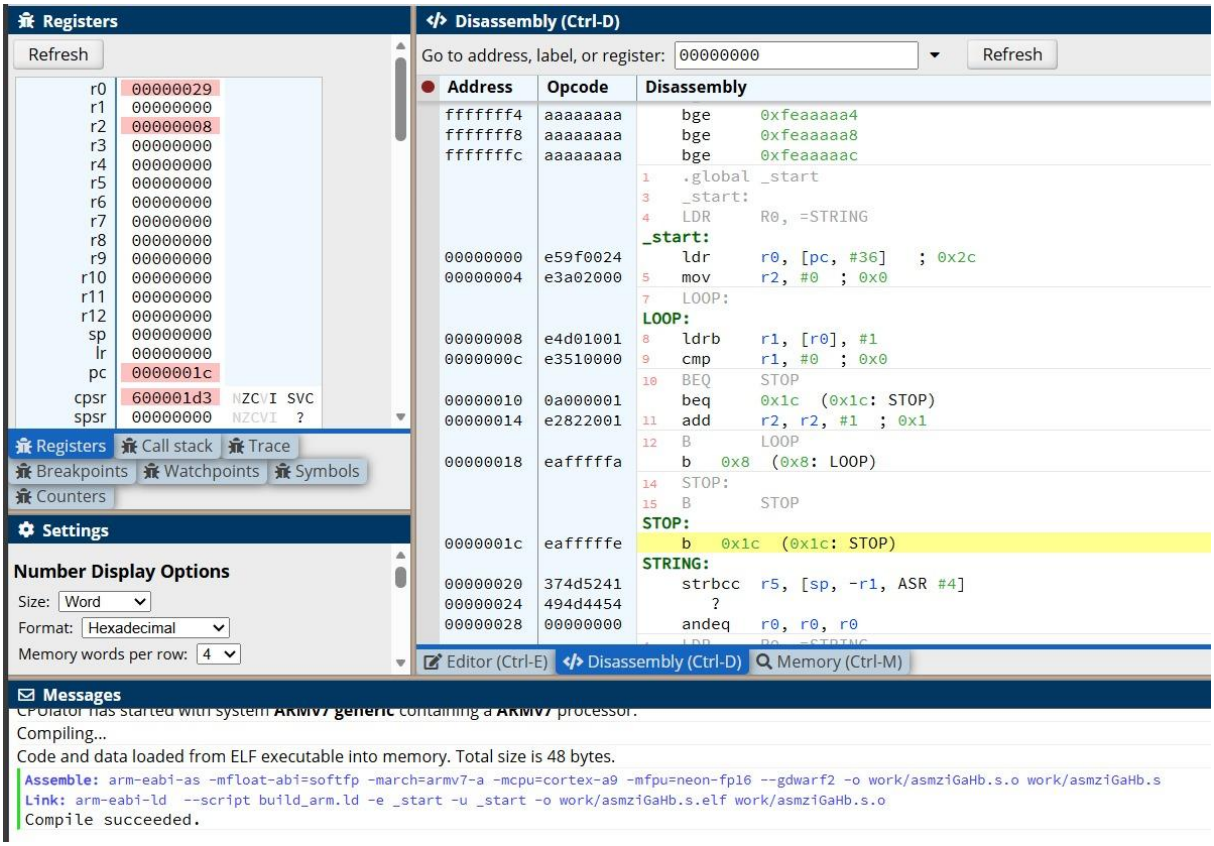
5	**ASSIGNMENT QUESTION 1**
	
	Write an ALP using conditional ARM instructions to sort an array of numbers using Bubble Sort Algorithm. Use STMFD to save registers before swap and LDMFD to restore the registers
	
	Program and Results Screenshot:
	
	 
	
	.global _start
	_start:
	LDR R0, =ARR
	MOV R1, #5
	OUTER_LOOP:
	SUBS R1, R1, #1
	BEQ STOP
	MOV R2, #0
	LDR R3, =ARR
	INNER_LOOP:
	CMP R2, R1
	BEQ OUTER_LOOP
	
	LDR R4, [R3]
	LDR R5, [R3, #4]
	CMP R4, R5
	BLE NO_SWAP

```
        STMFD   SP!, {R4, R5}
        STR     R5, [R3]
        STR     R4, [R3, #4]
        LDMFD   SP!, {R4, R5}
NO_SWAP:
        ADD     R3, R3, #4
        ADD     R2, R2, #1
        B       INNER_LOOP
STOP:
        B       STOP
ARR:
        .word   5, 1, 4, 2, 3
```

6 **ASSIGNMENT QUESTION 2**

Write an ALP using ARM7TDMI to find the length of string

Program and Results Screenshot:




```
.global _start

_start:
        LDR     R0, =STRING
        MOV     R2, #0

LOOP:
        LDRB    R1, [R0], #1
        CMP     R1, #0
        BEQ     STOP
```

```
        ADD     R2, R2, #1
        B       LOOP

STOP:
        B       STOP


STRING:
        .asciz  "ARM7TDMI"
```